

SLE-2: Profiling Report

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM044

Name: SANDESH PATIL

Division: A

Date: 22 September 2026

GitHub Link (optional): —

1. Algorithms / Versions Profiled

- Algorithm A: Breadth-First Search (BFS)
- Algorithm B: Depth-First Search (DFS, depth-limited to 30, with visited set)
- Problem used: 8-Puzzle — start state (1,2,3 / 4,_,6 / 7,5,8), goal state (1,2,3 / 4,5,6 / 7,8,_)

2. Profiling Method

- Tool used: Python's `time.perf_counter()` (via the `timeit`-style approach) for execution time, a manual counter for nodes expanded, and `cProfile` for a function-level breakdown.
- How I measured: Implemented BFS and DFS on the 8-puzzle state graph in Python. Each algorithm searches from the same start state to the same goal state. A `nodes_expanded` counter increments every time a state is popped from the frontier/stack and processed. Wall-clock time was measured around each full search call.
- Number of runs: 3 (average time reported below)

3. Results

Metric	BFS	DFS	Better?
Avg. Time (ms)	0.017	19.38	BFS
Nodes Expanded	3	17,742	BFS
Path Length (moves)	2 (optimal)	22	BFS

Short observation:

BFS solved the puzzle in about 0.017 ms, expanding only 3 nodes and returning the optimal 2-move solution. DFS took about 19.38 ms — roughly 1,140× slower — expanding 17,742 nodes and returning a much longer, non-optimal 22-move solution. `cProfile` confirmed the same picture: BFS made 42 function calls total, while DFS made 114,082 function calls, with most of the time spent inside the `neighbors()` function due to the sheer number of states visited.

4. Justification & Analysis

BFS is clearly better for this problem, on both time and solution quality. Because BFS explores the search space level by level, it is guaranteed to find the shortest path to the goal, which is why it stopped after expanding just 3 nodes — the goal was very close (2 moves away) and BFS reached it via the shortest route immediately. DFS, in contrast, commits to one branch and goes deep before backtracking, so even with a visited set it wandered through thousands of irrelevant states before stumbling on the goal, and the path it returned (22 moves) was far from optimal.

This matches the theory: BFS is complete and optimal for unweighted problems like the 8-puzzle (uniform cost per move), while DFS is only complete on finite spaces (with cycle/visited checking) and gives no optimality guarantee. If the problem size becomes larger (e.g., a start state that is 10+ moves from the goal, or the full 8-puzzle state space of 181,440 reachable states), BFS's memory usage would grow quickly since it stores the

entire frontier level by level, while DFS would use less memory but its runtime and solution quality would likely degrade even further, since the chance of a lucky short path drops as the search space grows.

5. AI Contribution Note

- AI tools used: Claude (Anthropic)
- What AI helped with: Writing the initial BFS/DFS Python implementation for the 8-puzzle, running the profiling (timeit-style timing, node counting, cProfile), and drafting this report structure/wording.
- What I did myself: Reviewed and understood the algorithm logic, selected the start state and problem setup, verified the results made sense against BFS/DFS theory, and prepared the final report with my own analysis and conclusions.

6. Conclusion

This exercise showed that theoretical properties of algorithms (BFS's optimality, DFS's lack of guarantees) show up clearly as measurable differences in real code — not just in textbooks. Profiling turned an abstract comparison into hard numbers: a 1,140× time difference and a very different solution quality. I learned that choosing the right search strategy matters a lot in practice, and that simple tools like a timer and a node counter are enough to make that difference visible and justifiable with data.